

Batch Gradient Descent

Contents

1. Overview	2
2. Gradient descent with many inputs	3
3. The derivatives	3
3.1 With two inputs	3
3.2 Any number of inputs	4
3.3 All derivatives at once	4
4. Batch gradient descent in code	5
5. How the result develops over epochs	5
6. Advantages and disadvantages	6
7. Summary	8
8. Key terms	8

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Gradient descent ([Note 57](#)).
- **Leads to:** Batch size in Keras ([Note 1020](#)).
- **Compare with:** Stochastic gradient descent ([Note 5](#)).

1. Overview

Key point: The three kinds of gradient descent differ only in how many rows they use for each update: all of them (batch), one (stochastic), or a small group (mini-batch).

Gradient descent comes in three main types. The update rule is the same in all three; they differ in how many training rows are used to compute each update (Figure 1).

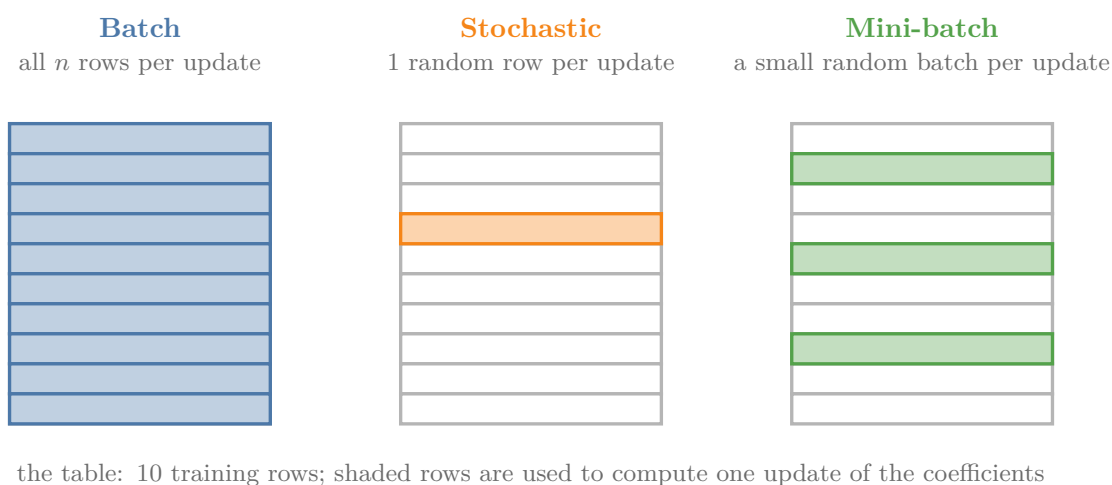


Figure 1: How many rows each type uses per update

Type	Rows per update	Updates per epoch (n rows)
Batch gradient descent	all n	1
Stochastic gradient descent (SGD)	1, chosen at random	n
Mini-batch gradient descent	a small random group, for example 32	n / batch size

The previous Note already used batch gradient descent, on one input. This Note extends it to any number of input columns, writes it with matrices, and codes it. The next two Notes cover the other two types.

2. Gradient descent with many inputs

Key point: With m inputs there are $m + 1$ coefficients, each with its own derivative; all are updated at the same step.

For multiple linear regression with m input columns, the model is

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_m x_{im}$$

and there are $m + 1$ coefficients to find. Batch gradient descent:

1. **Start** with any values, for example $\beta_0 = 0$ and every other $\beta_j = 1$.
2. **Each epoch:** compute the predictions for all rows, then the derivative of the loss with respect to every coefficient, then update all of them at once:

$$\beta_j \leftarrow \beta_j - \eta \frac{\partial L}{\partial \beta_j} \quad \text{for } j = 0, 1, \dots, m$$

3. **Repeat** for a fixed number of epochs.

The loss used here is the **mean** squared error, $L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$. Dividing by n keeps the size of the derivatives independent of how many rows there are, so the same learning rate works for small and large datasets.

3. The derivatives

Key point: The derivative for coefficient j is the errors multiplied by column j of the data, averaged, times -2 . The intercept uses a column of 1s.

3.1 With two inputs

Key point: Writing out the loss for two inputs shows the pattern: each coefficient's derivative weights the errors by its own input.

With two inputs, $\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$. Differentiating the mean squared error with the chain rule, as in the previous Note:

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\frac{\partial L}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{i1}$$

$$\frac{\partial L}{\partial \beta_2} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{i2}$$

3.2 Any number of inputs

Key point: The general rule: for coefficient j , multiply each row's error by that row's value in column j .

In words: for coefficient β_j , take each row's error, multiply it by the row's value in column j , add these up, and multiply by $-2/n$.

$$\frac{\partial L}{\partial \beta_j} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{ij}$$

The intercept fits the same rule if we imagine a column of 1s for it (as in the normal equation Note): multiplying by 1 changes nothing.

With numbers, for a tiny case of $n = 2$ rows with errors $(3, -1)$ and column j values $(2, 4)$:

$$\frac{\partial L}{\partial \beta_j} = -\frac{2}{2} (3 \times 2 + (-1) \times 4) = -(6 - 4) = -2$$

The derivative is negative, so this coefficient will increase.

3.3 All derivatives at once

Key point: All m derivatives together are one matrix product: $-\frac{2}{n} X^T (y - \hat{y})$.

Computing each derivative in a loop over rows and columns works but is slow in Python. The sum “errors times column j ” for every j at once is exactly a matrix product (Figure 2).

$$\underbrace{\frac{\partial L}{\partial \beta}}_{m \times 1} = -\frac{2}{n} \underbrace{X^T}_{m \times n} \underbrace{(y - \hat{y})}_{n \times 1}$$

one row per coefficient:
for coefficient j , the errors are
multiplied by column j of X , summed,
and divided by n

Figure 2: All derivatives as one matrix product

$$\frac{\partial L}{\partial \beta} = -\frac{2}{n} X^T (y - \hat{y})$$

X^T has one row per input column; multiplying it by the vector of n errors gives one number per column. Writing a computation as matrix operations instead of Python loops is called **vectorisation**. In the Notebook the vectorised derivative is about 13 times faster than the loop, even on this small dataset.

4. Batch gradient descent in code

Key point: Each epoch: predict all rows, compute the errors, compute the intercept's and the coefficients' derivatives, update both.

Python: Batch gradient descent for multiple linear regression.

```
class GDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.lr, self.epochs = learning_rate, epochs

    def fit(self, X, y):
        n = X.shape[0]
        self.intercept_ = 0.0
        self.coef_ = np.ones(X.shape[1])
        for _ in range(self.epochs):
            err = y - (X @ self.coef_ + self.intercept_)
            intercept_der = -2 * err.mean()
            coef_der = -2 * (X.T @ err) / n
            self.intercept_ -= self.lr * intercept_der
            self.coef_ -= self.lr * coef_der
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_
```

On the diabetes data (10 inputs, 353 training patients), with learning rate 0.5 and 1,000 epochs, training takes 0.02 seconds:

	Intercept	Test R^2
OLS (LinearRegression)	151.88	0.440
Batch gradient descent, 1,000 epochs	152.01	0.453

5. How the result develops over epochs

Key point: Training R^2 climbs towards the OLS value. Test R^2 peaks around epoch 1,000 slightly above OLS, then settles back towards it.

Figure 3 follows R^2 over 10,000 epochs.

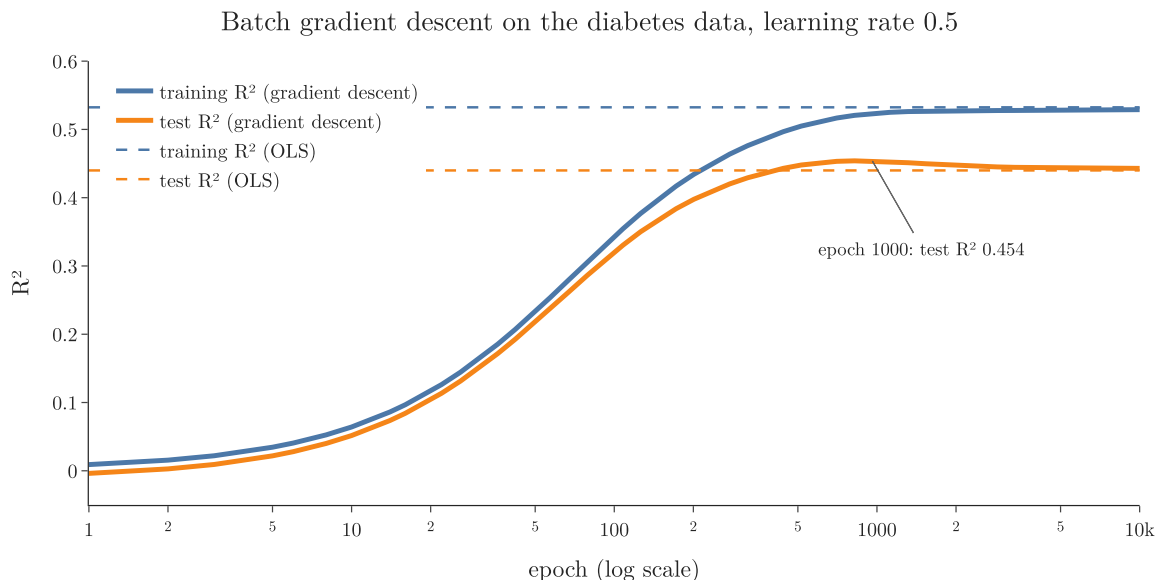


Figure 3: R^2 on training and test data over the epochs

- **Training R^2** (blue) rises steeply between epochs 10 and 500, then levels off just under the OLS value of 0.532. OLS gives the lowest possible training error, so gradient descent can approach it but never beat it.
- **Test R^2** (orange) peaks at 0.454 around epoch 1,000, slightly better than OLS's 0.440, then drifts back to 0.443 as the coefficients keep moving towards the OLS solution.

That gradient descent stopped early can do better on new data is not luck. Stopping before full convergence keeps the coefficients smaller and less tuned to the training data, which acts against overfitting. Stopping deliberately when the test score is best is called **early stopping**.

Extra: Even after 50,000 epochs the coefficients are still far from OLS's (the largest gap is 319), although R^2 is almost identical. Two blood measurements, s1 and s2, are strongly related (multicollinearity, from the assumptions Note), so the loss is a very long, flat valley along which many combinations of their coefficients are almost equally good. Gradient descent creeps along this valley, the same effect as with unscaled inputs in the previous Note.

6. Advantages and disadvantages

Key point: Batch gradient descent is smooth and stable, but each update needs every row, which is slow and memory-hungry on large data.

Advantages:

- **Stable:** every update uses the exact gradient of the whole training loss, so the loss goes down smoothly.

- **Vectorised:** the whole update is a few matrix products, which libraries compute very fast.

Disadvantages:

- **Slow on large data:** one update needs a pass over all n rows. With millions of rows, each step is expensive, and many steps are needed.
- **Memory:** the whole dataset must fit in memory at once for the matrix product.

These two problems are what stochastic and mini-batch gradient descent solve, in the next two Notes.

7. Summary

Item	Batch gradient descent
Rows per update	all n
Derivative of intercept	$-\frac{2}{n} \sum (y_i - \hat{y}_i)$
Derivative of coefficient j	$-\frac{2}{n} \sum (y_i - \hat{y}_i) x_{ij}$
All at once	$-\frac{2}{n} X^T (y - \hat{y})$
Diabetes data, 1,000 epochs	test R^2 0.453 (OLS 0.440)

- The three types of gradient descent differ only in how many rows feed each update.
- Each coefficient's derivative weights the errors by its own input column.
- Vectorised code computes every derivative with one matrix product.
- Batch gradient descent is stable but needs the whole dataset for every step.

8. Key terms

Term	Meaning
Batch gradient descent	Gradient descent that uses all training rows for every update
Stochastic gradient descent (SGD)	Gradient descent that uses one random row for every update
Mini-batch gradient descent	Gradient descent that uses a small random group of rows for every update
Mean squared error loss	The average squared error; its derivatives do not grow with the number of rows
Vectorisation	Writing a computation as operations on whole arrays instead of Python loops
Early stopping	Stopping training when the score on held-out data is best, before full convergence